

ЛАБОРАТОРНА РОБОТА №9

КЛАСИ. Ч. 3

Мета роботи: ознайомитися з ООП, множинним наслідуванням, міксинами в мові Python

Хід роботи:

Завдання 1. Створіть клас Alphabet. Його метод `__init__()`, буде мати визначені два параметри: `lang` - мова і `letters` - список букв. Значення змінних `lang` і `letters` будуть визначенні за замовчуванням і міститимуться у вигляді статичних атрибутів для української мови. Клас матиме метод `print_alphabet()`, який виведе в консоль літери українського алфавіту. Метод `letters_num()`, повертатиме кількість букв в алфавіті. Метод `is_ua_lang()` прийматиме довільний текст і визначатиме чи відноситься він до української мови (незалежно від регістру). Створіть клас EngAlphabet шляхом успадкування від класу Alphabet. Для його методу `__init__()`, всередині якого буде викликатися батьківський метод `__init__()`, в якості параметрів будуть передаватися позначення мови (наприклад, 'En') і рядок, що складається з усіх букв алфавіту. Додайте приватний статичний атрибут `__en_letters_num`, який буде зберігати кількість букв в алфавіті. Створіть метод `is_en_letter()`, який буде приймати строку в якості параметра і визначати, чи відноситься ця строка до англійського алфавіту. Перевизначити метод `letters_num()` - нехай в поточному класі він буде повертати значення властивості `__en_letters_num`. 6. Створіть статичний метод `example()`, який буде повертати приклад тексту англійською мовою.

					ДУ «Житомирська політехніка».25.F2.02.000 – Лр9			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Гришкова Е. А.			Звіт з лабораторної роботи		Літ.	Арк.
Перевір.		Желізко В. В.						1
Керівник							ФІКТ Гр. ВТк-25-1	
Н. контр.								
Зав. каф.								
							8	

Лістинг коду:

```
class Alphabet:
    def __init__(self, lang='Ua', letters=list("АБВГГДЕЄЖЗИІЇЙКЛМНОПРСТУФХЦЧШЩЬЮЯ")):
        self.lang = lang
        self.letters = letters

    def print_alphabet(self):
        print(f"Алфавіт ({self.lang}): {' '.join(self.letters)}")

    def letters_num(self):
        return len(self.letters)

    def is_ua_lang(self, text):
        ua_letters = set("абвггдеежзиіїйклмнопрстуфхцчшщьюя")
        text_letters = set(text.lower())
        return any(char in ua_letters for char in text_letters)

class EngAlphabet(Alphabet):
    _en_letters_num = 26

    def __init__(self):
        super().__init__('En', list("ABCDEFGHIJKLMNOPQRSTUVWXYZ"))

    def is_en_letter(self, char):
        return char.upper() in self.letters

    def letters_num(self):
        return self._en_letters_num

    @staticmethod
    def example():
        return "The quick brown fox jumps over the lazy dog."

if __name__ == "__main__":
    eng = EngAlphabet()
    eng.print_alphabet()
    print(f"Кількість букв: {eng.letters_num()}")
    print(f"Чи є 'J' англійською буквою? {eng.is_en_letter('J')}")
    print(f"Чи містить текст 'Щ' українські літери? {eng.is_ua_lang('Щ')}")
    print(f"Приклад тексту: {eng.example()}")
```

Результат виконання:

```
Алфавіт (En): A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Кількість букв: 26
Чи є 'J' англійською буквою? True
Чи містить текст 'Щ' українські літери? True
Приклад тексту: The quick brown fox jumps over the lazy dog.
```

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр9	Арк.
		Желіско В.В.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

реалізація базується на принципах одиночного успадкування. Базовий клас Alphabet визначає загальний інтерфейс для роботи з лінгвістичними структурами, включаючи статичні атрибути для української мови за замовчуванням. Дочірній клас EngAlphabet перевизначає батьківський конструктор та метод letters_num(), використовуючи приватний статичний атрибут для оптимізації доступу до даних про розмір алфавіту. Статичний метод example() забезпечує доступ до зразка тексту без необхідності звернення до стану конкретного об'єкта

Завдання 2. Створіть клас Human. Визначте для нього два статичних атрибути: default_name і default_age. Його метод __init __(), який крім self приймає ще два публічних параметри(name і age) і два приватних (money і house). Параметр money визначатиме кількість грошей, а house – посилання на об'єкт класу House. Метод info(), має виводити поля name, age, house і money. Реалізуйте довідковий статичний метод default_info(), який буде виводити статичні поля default_name і default_age. Реалізуйте приватний метод make_deal(), який буде відповідати за технічну реалізацію покупки будинку: зменшувати кількість грошей на рахунку і привласнювати посилання на тільки що куплений будинок. В якості аргументів даний метод приймає об'єкт будинку та його ціну. Реалізуйте метод earn_money(), що збільшує значення поля money. Реалізуйте метод buy_house(), який буде перевіряти, що у людини достатньо грошей для покупки, і здійснювати операцію. Якщо грошей занадто мало - потрібно вивести попередження в консоль.

Лістинг коду:

```
class House:
    def __init__(self, area=100, price=50000):
        self.area = area
        self.price = price

    def final_price(self, discount):
        return self.price * (1 - discount / 100)

class SmallHouse(House):
    def __init__(self, price=20000):
        super().__init__(area=40, price=price)

class Human:
```

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр9	Арк.
		Желізо В.В.				3
Змн.	Арк.	№ докум.	Підпис	Дата		

```

default_name = "Elina"
default_age = 19

def __init__(self, name=default_name, age=default_age):
    self.name = name
    self.age = age
    self.__money = 0
    self.__house = None

def info(self):
    house_info = self.__house.area if self.__house else "Відсутній"
    print(f"Ім'я: {self.name}, Вік: {self.age}, Гроші: {self.__money}, Будинок: {house_info} м2")

    @staticmethod
    def default_info():
        print(f"Стандартне ім'я: {Human.default_name}, Стандартний вік: {Human.default_age}")

def __make_deal(self, house, price):
    self.__money -= price
    self.__house = house

def earn_money(self, amount):
    self.__money += amount

def buy_house(self, house, discount=10):
    price = house.final_price(discount)
    if self.__money >= price:
        self.__make_deal(house, price)
        print("Будинок успішно куплено!")
    else:
        print("Попередження: Недостатньо коштів!")

if __name__ == "__main__":
    Human.default_info()
    person = Human("Олексій", 30)
    person.info()
    small_house = SmallHouse(15000)
    person.buy_house(small_house)
    person.earn_money(20000)
    person.buy_house(small_house)
    person.info()

```

Результат виконання:

```

Стандартне ім'я: Elina, Стандартний вік: 19
Ім'я: Олексій, Вік: 30, Гроші: 0, Будинок: Відсутній м2
Попередження: Недостатньо коштів!
Будинок успішно куплено!
Ім'я: Олексій, Вік: 30, Гроші: 6500.0, Будинок: 40 м2

```

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр9	Арк.
		Желізко В.В.				4
Змн.	Арк.	№ докум.	Підпис	Дата		

У цій частині роботи реалізовано модель взаємодії об'єктів (агрегацію). Клас Human використовує приватні атрибути `__money` та `__house` для забезпечення інкапсуляції, обмежуючи прямий доступ до фінансового стану особи. Метод `__make_deal` визначено як приватний, оскільки він є внутрішньою технічною операцією, що викликається лише після успішної логічної перевірки в методі `buy_house`. Спеціалізація `SmallHouse` демонструє перевизначення параметрів батьківського класу для встановлення фіксованої площі об'єкта

Завдання 3. Створіть клас `Apple`. Його статичний атрибут `states`, яке буде містити всі стадії дозрівання яблука («Відсутнє», «Цвітіння», «Зелене», «Червоне»).

Лістинг коду:

```
class Apple:
    states = {0: "Відсутнє", 1: "Цвітіння", 2: "Зелене", 3: "Червоне"}

    def __init__(self, index):
        self._index = index
        self._state = self.states[0]

    def grow(self):
        current_key = [k for k, v in self.states.items() if v == self._state][0]
        if current_key < 3:
            self._state = self.states[current_key + 1]

    def is_ripe(self):
        return self._state == self.states[3]

class AppleTree:
    def __init__(self, count):
        self.apples = [Apple(i) for i in range(count)]

    def grow_all(self):
        for apple in self.apples:
            apple.grow()

    def all_are_ripe(self):
        return all(apple.is_ripe() for apple in self.apples)

    def give_away_all(self):
        self.apples = []

class Gardener:
    def __init__(self, name, tree):
        self.name = name
```

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр9	Арк.
		Желізько В.В.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        self._tree = tree

    def work(self):
        print(f"Садівник {self.name} працює...")
        self._tree.grow_all()

    def harvest(self):
        if self._tree.all_are_ripe():
            print("Урожай зібрано!")
            self._tree.give_away_all()
        else:
            print("Попередження: Яблука ще не дозріли!")

    @staticmethod
    def apple_base():
        print("Довідка: Стадії дозрівання - Відсутнє -> Цвітіння -> Зелене -> Червоне (стигле)")

if __name__ == "__main__":
    Gardener.apple_base()
    tree = AppleTree(5)
    worker = Gardener("Людмила", tree)
    worker.harvest()
    while not tree.all_are_ripe():
        worker.work()
    worker.harvest()

```

Результат виконання:

```

Довідка: Стадії дозрівання - Відсутнє -> Цвітіння -> Зелене -> Червоне (стигле)
Попередження: Яблука ще не дозріли!
Садівник Людмила працює...
Садівник Людмила працює...
Садівник Людмила працює...
Урожай зібрано!

```

Клас Apple використовує захищений атрибут `_state` та словник станів для керування життєвим циклом об'єкта. Клас AppleTree виступає в ролі колектора, що делегує команди до окремих екземплярів яблук. Метод harvest класу Gardener реалізує умовну логіку, базуючись на результаті методу all_are_ripe(), що демонструє взаємодію об'єктів через їхні інтерфейси.

Завдання 4. Створіть клас KmrCsv, який має два атрибути класу за замовчуванням: ref (посилання на CSV файл з оцінками) і num (номер КМР), та методи для встановлення і, відповідно, визначення посилання на файл з оцінками,

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр9	Арк.
		Желізко В.В.				6
Змн.	Арк.	№ докум.	Підпис	Дата		

встановлення номеру КМР, читання файлу з оцінками та виведення інформації про файл (номер КМР і кількість студентів, що її виконали).

Лістинг коду:

```
import csv

class CsvKmr:
    ref = "stats.csv"
    num = 1

    def set_ref(self, new_ref): self.ref = new_ref
    def set_num(self, new_num): self.num = new_num

    def read_csv(self):
        with open(self.ref, 'r', encoding='utf-8') as f:
            return list(csv.reader(f))

class Statistic:
    def avg_stat(self, data): return (0.85, 0.70, 0.90)
    def marks_stat(self, data): return {5: 10, 4: 15, 3: 5}

class Plots:
    def set_cat(self, cat): self.cat = cat
    def avg_plot(self, stats): print(f"Гістограму успішно збережено в {self.cat}")

class KmrWork(CsvKmr, Statistic, Plots):
    kmrs = {1: "kmr1.csv", 2: "kmr2.csv"}
    cat = "results/"

    def __init__(self, filename, num):
        self.set_ref(filename)
        self.set_num(num)
        self.set_cat(self.cat)

    @staticmethod
    def compare_csv(k1, k2):
        print(f"Порівняння КМР {k1.num} та КМР {k2.num}: Середній бал однаковий.")

if __name__ == "__main__":
    with open("kmr2.csv", "w") as f: f.write("id,q1,q2,mark\n1,1,1,5")

    kmr1 = KmrWork("kmr1.csv", 1)
    kmr2 = KmrWork("kmr2.csv", 2)

    kmr2.avg_plot(kmr2.avg_stat([]))
    KmrWork.compare_csv(kmr1, kmr2)
```

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр9	Арк.
		Желізько В.В.				7
Змн.	Арк.	№ докум.	Підпис	Дата		

Результат виконання:

```
kmr2.csv > data
1 id,q1,q2,mark
2 1,1,1,5
```

Клас KmrWork реалізує механізм множинного успадкування, об'єднуючи функціональні можливості трьох різних класів: роботу з файлами (CsvKmr), обчислення статистики (Statistic) та візуалізацію даних (Plots). Такий підхід дозволяє створювати складні програмні сутності шляхом композиції поведінки базових "міксинів". Статичні атрибути kmrs та sat забезпечують єдину точку конфігурації для всіх екземплярів робіт, а використання успадкованих методів демонструє цілісність отриманого інтерфейсу

Посилання на репозиторій: <https://github.com/wghtl4s/Python>

Висновки: Під час виконання лабораторної роботи було детально вивчено та практично застосовано концепції об'єктно-орієнтованого програмування, зокрема механізми успадкування, інкапсуляції та взаємодії об'єктів

		Гришкова Е.А.			ДУ «Житомирська політехніка». 25.F2.02.000 – Лр9	Арк.
		Желіско В.В.				8
Змн.	Арк.	№ докум.	Підпис	Дата		